

ROADMAP INTENSIVO — De Principiante a Technical Co-Founder

6 Semanas | Mayo 5 — Junio 16, 2026

Correcciones de arquitecto antes de empezar

"Crackear githubs" → el término profesional es **reverse-engineering de repos open source**: clonar, leer el código, entender la arquitectura, modificarlo. Es una de las mejores formas de aprender. Eso es lo que harás.

"Vibe coding" → en contexto profesional se llama **AI-assisted development**. La habilidad real no es solo usar IA para codear — es saber qué pedirle, validar lo que genera, y entender por qué funciona. Ahí está la diferencia entre un vibe coder amateur y uno senior.

MCP → correcto en concepto, pero es una capa de integración, no un framework de agentes. Lo usarás como herramienta, no como base.

Stack definitivo del roadmap

Capa	Tecnología	Por qué
Lenguaje base	Python 3.11+	Ecosistema IA más maduro
APIs & Backend	FastAPI + Pydantic	Estándar de la industria para microservicios IA
Automatización	n8n self-hosted	Ya lo conoces, es production-ready
IA	Claude API (Anthropic)	Mejor para agentes y razonamiento
Base de datos	Supabase (Postgres)	SaaS-ready, auth incluida
Agentes	LangGraph	Más robusto que LangChain para multi-agente
MCP	Claude MCP SDK	Integración real de herramientas
Deploy	Railway + Docker	Simple, profesional, barato
Vibe Coding	Claude Code CLI	Tu IDE principal de aquí en adelante

SEMANA 1 — Mayo 5-11: Foundation Real

Objetivo: Eliminar los gaps que te frenan. Salir con Python sólido orientado a APIs y un workflow mental de arquitecto.

Lunes 5 — Python como arquitecto, no como tutorial

Mañana (5am-8am) - Instalar entorno: `pyenv` + Python 3.11 + `uv` (reemplaza pip, es 10x más rápido) - Leer y replicar: estructuras de datos reales — dict anidados, list comprehensions, funciones con type hints - Tarea: script que consume la API de OpenWeather, parsea JSON y guarda en archivo `.json` local

Tarde (2h) - Estudiar: `Pydantic v2` — modelos de datos, validación, por qué toda API sería lo usa - Construir: modelo Pydantic que valide una factura (cliente, items, total, fecha)

Por qué: Pydantic es el lenguaje común entre FastAPI, LangChain, y agentes de IA. Sin esto, no puedes leer código de producción.

Martes 6 — APIs como ciudadano de primera clase

Mañana - Aprender: `httpx` (reemplaza `requests` en código async), manejo de errores HTTP, retry logic - Construir: cliente Python que consulta 3 APIs distintas y normaliza los datos en un esquema Pydantic común

Tarde - Estudiar: autenticación real — API Keys, Bearer tokens, OAuth2 básico - Tarea: integrar la API de Notion o Airtable y leer/escribir registros desde Python

Miércoles 7 — Git como profesional

Mañana - Workflow real: `git branch`, `git rebase`, `git stash`, pull requests, conventional commits - Clonar un repo open source real (sugerido: `instructor` de Jason Liu — librería Pydantic + IA) - Leer su estructura: cómo organiza módulos, tests, documentación

Tarde - Crear tu repo personal `ai-arsenal` en GitHub — aquí vivirán todos tus proyectos del roadmap - Primer commit: tu cliente de APIs del martes, bien estructurado

Por qué del repo personal: En 6 semanas tendrás un portafolio real que mostrar a clientes.

Jueves 8 — n8n: de usuario a arquitecto

Mañana - Instalar n8n self-hosted con Docker en tu máquina local (no usar cloud) - Estudiar: diferencia entre trigger, node, webhook, y credential en n8n - Reverse-engineering: importar un workflow complejo de la comunidad n8n, entender cada nodo

Tarde - Construir: workflow que recibe un webhook POST con datos JSON, los procesa y envía un email formateado - Documentar en tu repo: diagrama del flujo + descripción de cada nodo

Viernes 9 — JSON → estructura de datos real

Mañana - Profundizar: `jq` en terminal para manipular JSON sin Python - JSONPath, JSON Schema, diferencia entre JSON y JSONL - Tarea: tomar un PDF de factura (usa cualquier factura real), extraer datos manualmente y modelarla en JSON estructurado

Tarde - Construir: pipeline Python que lee 5 JSONs diferentes, los normaliza a un esquema común y genera un CSV de resumen

Sábado 10 — Proyecto integrador semana 1

Todo el día

Proyecto: "Extractor de documentos básico" - Input: PDF o imagen de documento - Proceso: Python parsea el archivo → estructura JSON → valida con Pydantic - Output: archivo CSV limpio

No uses IA aún — hazlo con `pdfplumber` o `pypdf2`. El objetivo es entender el problema antes de resolverlo con IA.

Domingo 11 — Revisión y arquitectura mental

- Documentar qué aprendiste y qué no entendiste
- Leer: "The Twelve-Factor App" (12factor.net) — son los principios que sigue todo SaaS profesional
- Preparar entorno semana 2

SEMANA 2 — Mayo 12-18: IA Integration Real

Objetivo: Integrar Claude API directamente. Entender prompt engineering como disciplina técnica.

Lunes 12 — Claude API desde cero

Mañana - Instalar: `anthropic` SDK Python - Leer la documentación oficial de mensajes, tokens, y modelos (no tutoriales de YouTube) - Construir: script que recibe texto libre y devuelve JSON estructurado usando Pydantic + Claude

```
response = client.messages.create(  
    model="claude-sonnet-4-6",  
    messages=[...],
```

```
tools=[factura_schema]
)
```

Tarde - Estudiar: Tool Use (function calling) en Claude — esto es lo que hace a Claude útil en agentes - Construir: función que dado texto de una factura, extrae campos estructurados via tool use

Martes 13 — Prompt Engineering profesional

Mañana - Estudiar: la diferencia entre zero-shot, few-shot, chain-of-thought, y system prompts - Principio clave: **el prompt es código** — debe ser versionado, testeado, y optimizado - Construir: 3 versiones del mismo prompt para extracción de datos, medir cuál es más precisa

Tarde - Crear tu `prompt_library/` en el repo — estructura de carpetas por dominio - Cada prompt debe tener: objetivo, versión, variables de entrada, output esperado

Por qué: Las empresas que ganan con IA no tienen mejor modelo — tienen mejores prompts.

Miércoles 14 — Pipeline IA real

Mañana - Construir el pipeline que describiste: `JSON → IA analista → IA generador de reporte` - Usar dos llamadas a Claude: una para analizar datos, otra para redactar el reporte - Output: archivo Markdown o HTML con el reporte

Tarde - Optimizar: añadir prompt caching (reduce costos hasta 90% en llamadas repetidas) - Medir tokens usados en cada llamada, calcular costo real

Jueves 15 — n8n + Claude integrados

Mañana - Conectar n8n con Claude API via HTTP Request node - Construir: workflow que recibe email con adjunto → extrae texto → Claude analiza → responde con resumen

Tarde - Añadir: manejo de errores en el workflow (qué pasa si Claude falla, si el email no tiene adjunto) - Documentar el workflow con anotaciones en n8n

Viernes 16 — Structured Outputs mastery

Mañana - Estudiar: `instructor` library (el repo que clonaste semana 1) — hace Pydantic + IA trivial - Reescribir tu extractor de facturas usando `instructor` — debería quedar en la mitad del código

Tarde - Explorar: cómo `instructor` maneja reintentos automáticos cuando la IA genera JSON inválido - Construir: extractor que funcione con 5 formatos diferentes de factura peruana

Sábado 17 — Proyecto integrador semana 2

Proyecto: "Asistente de documentos logísticos"

Esto es una versión real del producto que cotizaste a TRACTO KCAR: - Input: PDF de guía de remisión o factura - Claude extrae: número, fecha, origen, destino, items, pesos - Output: fila en Google Sheets o CSV - Trigger: carpeta de Google Drive o webhook

Stack: Python + Claude API + instructor + n8n (para el trigger)

Domingo 18 — Deep dive en costos y límites

- Calcular: si TRACTO KCAR procesa 100 docs/día, ¿cuánto cuesta en tokens?
- Estudiar: rate limits de Claude API, cómo manejarlos con retry exponencial
- Actualizar tu propuesta técnica con números reales

SEMANA 3 — Mayo 19-25: Agentes Reales

Objetivo: Construir tu primer agente funcional. Entender la diferencia entre un chatbot y un agente.

Lunes 19 — Qué es un agente (definición técnica)

Mañana - Leer: paper "ReAct: Synergizing Reasoning and Acting in Language Models" - Concepto clave: **un agente = LLM + herramientas + loop de decisión** - No es magia — es un while loop donde el LLM decide qué tool usar en cada iteración

Tarde - Construir: agente mínimo en Python puro (sin frameworks) que: 1. Recibe una pregunta 2. Decide si buscar en web o responder directo 3. Si busca, usa una tool, obtiene resultado, responde

Martes 20 — LangGraph: agentes con estado

Mañana - Instalar y estudiar LangGraph (no LangChain — LangGraph es el sucesor para agentes complejos) - Concepto: grafo de estados donde cada nodo es una acción del agente - Construir: agente simple con LangGraph que tiene 3 nodos: analyze → search → respond

Tarde - Estudiar: cómo LangGraph maneja memoria de corto plazo (dentro de una conversación) - Añadir memoria al agente: que recuerde lo que dijo en mensajes anteriores

Miércoles 21 — Tools y MCP

Mañana - Estudiar MCP correctamente: es un protocolo para que agentes accedan a herramientas externas de forma estandarizada - Instalar Claude Desktop + MCP filesystem server - Construir tu primer MCP server simple en Python: una tool que lee archivos de una carpeta

Tarde - Conectar tu MCP server con Claude Desktop - Probar: pedirle a Claude que lea y analice archivos de tu máquina via MCP

Por qué MCP importa: Es el estándar que está ganando para integración de herramientas. En 2026 todos los agentes serios lo usan.

Jueves 22 — Sub-agentes (lo que llamas "skills")

Mañana - Concepto correcto: en LangGraph se llaman **subgraphs** o en Claude se implementan como **tool calls a otros agentes** - Patrón: orquestador → delega tarea a agente especialista → recibe resultado → continúa

Tarde - Construir: sistema con 2 agentes - Agente 1 (orquestador): recibe pedido del usuario - Agente 2 (especialista): extrae datos de documentos - Orquestador llama al especialista como si fuera una tool

Viernes 23 — Agente con memoria persistente

Mañana - Estudiar: diferencia entre memoria de corto plazo (conversación) y largo plazo (base de datos) - Integrar Supabase: crear tabla `agent_memory` donde el agente guarda contexto entre sesiones

Tarde - Construir: agente que recuerda preferencias del usuario entre conversaciones distintas

Sábado 24 — Proyecto: Sistema de agentes para Musuq

Todo el día

Proyecto: "Client Intelligence Agent" - Agente orquestador que gestiona clientes de Musuq - Sub-agente 1: busca información del cliente en Supabase - Sub-agente 2: analiza el estado del proyecto (etapas) - Sub-agente 3: genera reporte de progreso semanal - Interface: CLI por ahora, web después

Este es el sistema de tracking de clientes que necesitas para Musuq.

Domingo 25 — Revisión arquitectónica

- Con tu socio (el arquitecto DevOps): revisar lo construido, identificar problemas de arquitectura
- Documentar: diagrama de flujo de cada agente construido

- Definir: ¿qué datos necesita cada agente? ¿Dónde viven esos datos?

SEMANA 4 — Mayo 26 — Junio 1: Backend SaaS

Objetivo: Construir el backend de un mini SaaS funcional. FastAPI + Supabase + auth real.

Lunes 26 — FastAPI desde cero

Mañana - Instalar FastAPI + Uvicorn - Construir: API con 5 endpoints CRUD para gestión de clientes - Usar Pydantic para validar todos los inputs/outputs

Tarde - Añadir: Supabase como base de datos (reemplaza SQLite de los tutoriales) - Entender: connection pooling, por qué importa en producción

Martes 27 — Auth real

Mañana - Integrar Supabase Auth en FastAPI - Implementar: registro, login, JWT tokens, middleware de autenticación - Patrón: cada endpoint verifica el token antes de responder

Tarde - Añadir: Row Level Security en Supabase (cada usuario solo ve sus datos) - Construir: endpoint que retorna solo los clientes del usuario autenticado

Miércoles 28 — API para tu agente

Mañana - Exponer tu agente de Musuq (semana 3) como endpoint FastAPI - `POST /agent/query` → recibe mensaje del usuario → agente procesa → retorna respuesta

Tarde - Añadir: historial de conversaciones guardado en Supabase - Websockets básicos para respuesta en streaming (el usuario ve el texto aparecer)

Jueves 29 — Docker y deploy

Mañana - Containerizar tu FastAPI app: Dockerfile mínimo funcional - `docker-compose.yml` con FastAPI + Supabase local - Probar: que todo funcione igual dentro del container

Tarde - Deploy en Railway: conectar repo GitHub → deploy automático - Configurar variables de entorno de forma correcta (nunca hardcodear API keys)

Viernes 30 — Webhooks y eventos

Mañana - Construir: sistema de webhooks salientes (tu app notifica a otros sistemas cuando algo cambia) - Construir: webhook entrante seguro (verificación de firma HMAC)

Tarde - Conectar: cuando se crea un cliente en tu SaaS → webhook a n8n → n8n hace acciones automatizadas

Sábado 31 — Proyecto: API completa

Proyecto: "Musuq Client Tracker API" - Auth con Supabase - CRUD de clientes con etapas (pipeline) - Metas semanales por usuario - Endpoint de agente IA para consultas - Dockerizado y deployado en Railway

Este es el backend del CRM interno que diseñaste al inicio.

Domingo 1 de Junio — Seguridad básica

- Rate limiting en FastAPI
- Input sanitization (nunca confiar en datos del cliente)
- CORS configurado correctamente
- Revisar: OWASP Top 10 — las 10 vulnerabilidades más comunes

SEMANA 5 — Junio 2-8: Vibe Coding Mastery

Objetivo: Dominar AI-assisted development como metodología profesional.

Lunes 2 — Claude Code como IDE

Mañana - Instalar Claude Code CLI (ya lo tienes en este repo de Musuq) - Aprender: cómo escribir prompts efectivos para código (contexto, restricciones, output esperado) - Ejercicio: tomar un feature del Musuq Client Tracker, implementarlo 100% con Claude Code

Tarde - Estudiar tu propio código generado: ¿lo entiendes? ¿Cambiarías algo? - Principio: **nunca deployar código que no puedes explicar**

Martes 3 — Debugging con IA

Mañana - Metodología: cómo describir un bug a Claude para obtener la solución más rápido - Estructura: contexto → comportamiento esperado → comportamiento actual → stack trace - Ejercicio: romper intencionalmente 3 partes de tu código y debuggear con IA

Tarde - Estudiar: cómo leer stack traces en Python y FastAPI - Construir: sistema de logging estructurado (JSON logs) para tu API

Miércoles 4 — Reverse Engineering de repos

Mañana - Clonar: `open-webui` (UI para modelos de IA, 40k+ stars) - Analizar: estructura de carpetas, cómo maneja auth, cómo conecta con APIs de IA - Extraer: 2-3 patrones que puedas usar en tu propio código

Tarde - Clonar: `langflow` (builder visual de agentes) - Analizar: cómo representa agentes como JSON, cómo los ejecuta

Jueves 5 — Prompting para arquitectura

Mañana - Aprender: cómo usar IA para diseñar arquitecturas, no solo escribir código - Ejercicio: describir un problema de negocio a Claude y pedirle que diseñe la arquitectura completa - Evaluar críticamente: ¿es la arquitectura correcta? ¿Dónde falla?

Tarde - Construir: tu propio template de "architecture prompt" — cómo le describes problemas a la IA

Viernes 6 — Testing con IA

Mañana - Escribir tests para tu API usando pytest - Usar Claude para generar los tests: describe el comportamiento esperado, IA genera el test - Entender: qué es un test unitario vs de integración

Tarde - Configurar: GitHub Actions que corre tests en cada push - Tu repo ahora tiene CI/CD básico

Sábado 7 — Demo Day Prep

Construir demo para cliente ficticio: - Escenario: empresa logística que necesita procesar guías de remisión - Demo en vivo: subir PDF → sistema extrae datos → genera reporte → envía por email - Stack completo: n8n + Claude API + FastAPI + Supabase

Grabar el demo. Este video entra a tu portafolio.

Domingo 8 — Revisión con socio arquitecto

- Presentar todo lo construido al socio
- Recibir feedback técnico real
- Identificar los 3 problemas más importantes a resolver la semana siguiente

SEMANA 6 — Junio 9-15: Los 3 Proyectos Finales

Objetivo: Terminar con 3 proyectos deployados y documentados.

Proyecto 1 (Lunes-Martes): Automatización con IA

"Document Intelligence Pipeline" - Input: cualquier documento (PDF, imagen, Excel)
- Agente extrae, clasifica y estructura los datos - Output: Supabase + notificación -
Deploy: Railway + n8n cloud - Precio de venta sugerido: S/ 2,500-3,500

Proyecto 2 (Miércoles-Jueves): Sistema de Agentes

"Research & Report Agent" - Input: tema o empresa a investigar - Agente 1: busca información en web - Agente 2: analiza y filtra - Agente 3: redacta reporte profesional en PDF - Use case: due diligence de clientes, análisis de mercado

Proyecto 3 (Viernes-Sábado): Mini SaaS

"Musuq Client OS" — el CRM interno que necesitan - Auth por usuario (jefe vs miembro) - Pipeline de clientes con etapas - Metas semanales con reset automático - Reuniones adjuntas a clientes - Dashboard diferente por rol - Deploy: Railway + dominio propio

Domingo 15 — Documentación y portafolio

- README profesional para cada proyecto
- Diagrama de arquitectura (usa Excalidraw)
- Video demo de cada uno (max 3 min)
- Publicar en GitHub con descripción clara

Métricas de éxito al finalizar las 6 semanas

Habilidad	Nivel inicial	Nivel objetivo
Python APIs	Básico	Intermedio-alto
IA Integration	Ninguno	Producción
Agentes	Conceptual	Funcional
FastAPI Backend	Ninguno	Deployado
n8n avanzado	Básico	Arquitecto
Vibe Coding	Amateur	Metodología estructurada
Criterio técnico	Bajo	Puede asesorar clientes

Recursos (solo los esenciales)

Recurso	Para qué
docs.anthropic.com	Claude API — leer completo
fastapi.tiangolo.com	FastAPI — documentación oficial
langchain-ai.github.io/langgraph	LangGraph
supabase.com/docs	Supabase
n8n.io/workflows	Templates de workflows reales
github.com/jxnl/instructor	Structured outputs con IA

No: Udemy, YouTube tutoriales de 10 horas, libros de texto. Solo documentación oficial y repos reales.

Una verdad de arquitecto

En 6 semanas no serás senior. Pero serás algo más valioso para tu startup: **alguien que puede construir MVPs funcionales, hablar con criterio técnico con clientes, y complementar a un arquitecto DevOps sin ser una carga**. Eso tiene precio de mercado real en Lima y LATAM.

El diferencial no es cuánto sabes — es qué puedes entregar.